

The entire optimised code for this is around 2000 lines! The reason for this is that ClojureScript provides an entire framework that is working behind the scenes to improve the way JavaScript works, and unfortunately that framework isn't going to go away if you don't use it. In short, ClojureScript is for creating full-fledged web applications – not for “Hello World” examples—and that is where it will shine.

haXe

This language is more than just a JavaScript alternative. Instead, JavaScript is one of the outputting formats. Code written in haXe can be converted to source code (or bytecode) for Flash (for both AVM1, or Flash 6 to 8; and AVM2, Flash 9 onwards), Neko, PHP, ActionScript, JavaScript, C++, Java, and C#. Java and C# are still in development.

This essentially means that you could write your entire web application in haXe! From the server-side code (running as PHP, on the Neko VM or even C++), the client side JavaScript, the client-side Flash content, and even the iPhone app (in C++) – why not!

With code shared between server and client you can easily pass data from client to server as the same classes can be used on both side. Code can be shifted between client and server with ease before finding the perfect mix of client and server performance. Over and above the benefits of one language across platforms, haXe brings type checking to PHP and JavaScript (which lack this feature), a powerful package and class syntax and much more.

haXe will be familiar to those already know ActionScript / JavaScript as it is based on ECMAScript 4, at one time a successor for JavaScript, and the basis for ActionScript. So let's take a look at a sample, the same example again:

```
package;
import js.Dom
class Main {
    public static function main() {
        new Main();
        js.Lib.document.write(Main.square(10).join(", "));
    }
    public static function square(n : Int) : Array < Int > {
        var i : Int = 0;
        var result : Array < Int > = new Array < Int > ();
        for (i in i...n) {
            result.push(i * i);
        }
        return result;
    }
}
```

The main function of the application class will be called automatically on launch. Here too, the code is rather too long (over 300 lines). However, with a full fledged application, it can be efficient.

Dart

Finally we come to Dart, the new JavaScript alternative introduced by Google. The fact that Google backs it, makes Dart quite important; not only because they created it, but because they want it to eventually become a language supported in all browsers – directly, without translation to JavaScript. It is a little ironic a Google employee proposed making the script tag's `type="text/javascript"` optional, because JavaScript was the only scripting language supported by browsers.

Dart adds a lot to JavaScript while keeping a similar syntax. A JavaScript developer will not find it difficult to understand Dart code or write it. The syntax and feature set of Dart seems to have been

*Opa

>>Opa is an alternative to the server and client stack of your web application. It's designed specifically for making web applications that will run on multiple clients, and servers and includes a programming language, a server and even a database.

To create a web application, you write code in Opa, which compiles to an application. That application now handles the server side (handling requests,

accessing the database etc.) and client side (the user interface, etc) requests.

Both the code that has to run on the server, and the code that has to run on the client will be written as part of the same application, and Opa will automatically create the HTML-based interface with jQuery for interactivity. The part of your code that goes to the server and client is managed automatically!

influenced by the need to make code easier to optimise. Dart makes it simpler to create web applications, rather than simple scripts.

Dart includes Classes and interfaces for more structures web applications, and an optional types that can help make the code easier to understand and optimise. Dart applications have a `main()` function that is executed to begin with. Dart also has a system of libraries that will allow easy reuse of code.

Currently the compiler for converting Dart code to JavaScript has to be built from source, as there is not prebuilt binary for any platform. However, a light Eclipse based IDE called “Dart Editor” is available that does the compilation for you.

Now let us take a look at the same code once again from a Dart perspective:

```
#import('dart : dom');
class Example {
    void run() {
        HTMLDocument doc = window.document;
        doc.write(square(10));
    }
    List < int > square(int n) {
        var result = new List < int > ();
        for (int i = 0; i < n; i++) {
            result.add(i * i);
        }
        return result;
    }
}
void main() {
    new Example().run();
}
```

This could obviously have been done without a class, but this can serve as an example. Now, if you were shocked that haXe produces a 300-line JavaScript, and Clojure produced 2000 lines of JavaScript, we hope you're sitting down now; because this above code translates to 167472 lines of JavaScript and weighs around 6MB! This code includes the JavaScript implementation of all of Dart's features and libraries. Dart is still in development and this is clearly very un-optimised code even if it is for the entire framework. Of course when / if it is supported in the browser you will need to only include the above code. Printing even the relevant bits of JavaScript here will be a waste of space, but you are free to try the above sample in the IDE to see the code it produces. >

[Join us on Facebook at dvwx.in/dvwx-fb](http://devworx.in/dvwx-fb) and post your comments.